

KubeQuest — Questions probables du jury & réponses

Astuce : réponds toujours en 3 temps — **ce qu'on a fait** → **pourquoi** → **limite/amélioration**. Ça montre de la maîtrise, pas de la récitation.

0. Questions de base (fondamentaux) — à connaître par cœur

Concepts généraux

- **C'est quoi Kubernetes ?** Un orchestrateur de conteneurs : il déploie, met à l'échelle et surveille automatiquement des applications conteneurisées sur plusieurs machines.
- **C'est quoi un conteneur ?** Un paquet léger et isolé contenant une application + ses dépendances ; il partage le noyau de la machine hôte.
- **Conteneur vs machine virtuelle ?** La VM virtualise le matériel (OS complet, lourde, démarre en minutes) ; le conteneur virtualise l'OS (léger, démarre en secondes).
- **C'est quoi Docker ?** Un outil pour construire et lancer des conteneurs. Ici le runtime du cluster est **containerd** (même standard OCI).
- **C'est quoi une image ?** Un modèle figé en lecture seule à partir duquel on démarre des conteneurs (app + librairies).
- **C'est quoi un registre (registry) ?** Un dépôt d'images (Docker Hub ou privé). Chez nous : pas de registre, l'image est chargée directement dans chaque nœud.
- **C'est quoi YAML ?** Un format texte lisible pour décrire la configuration ; les manifestes Kubernetes sont écrits en YAML.
- **C'est quoi le cloud / AWS ?** Des ressources informatiques à la demande. Ici, 4 machines virtuelles EC2 sur AWS.

Objets Kubernetes

- **C'est quoi un pod ?** La plus petite unité déployable : un (ou plusieurs) conteneur(s) qui partagent réseau et stockage.
- **C'est quoi un nœud (node) ?** Une machine du cluster qui exécute des pods.
- **C'est quoi le control-plane ?** Le « cerveau » du cluster : il décide et pilote (API server, scheduler, controller-manager).
- **C'est quoi le kubelet ?** L'agent présent sur chaque nœud qui lance et surveille les pods.
- **C'est quoi etcd ?** La base de données clé-valeur qui stocke tout l'état du cluster.
- **C'est quoi un namespace ?** Une séparation logique pour organiser les ressources (ex. `monitoring`, `laravel-app-prod`).
- **C'est quoi un Deployment ?** Il gère un ensemble de pods identiques (réplicas) et leurs mises à jour.
- **C'est quoi un ReplicaSet ?** Il garantit qu'un nombre donné de réplicas tourne en permanence.
- **Deployment vs StatefulSet ?** Deployment = pods interchangeables (sans état) ; StatefulSet = identité et stockage stables (ex. base de données).
- **C'est quoi un Service ?** Une adresse réseau stable + répartition de charge devant un groupe de pods.
- **C'est quoi un Ingress ?** La règle d'entrée HTTP(S) qui route le trafic externe vers les services par nom de domaine.
- **C'est quoi un label / un selector ?** Un label est une étiquette clé/valeur sur une ressource ; un selector sélectionne les ressources par leurs labels.
- **ConfigMap vs Secret ?** ConfigMap = configuration non sensible ; Secret = données sensibles (mots de passe, clés).
- **C'est quoi kubectl ?** L'outil en ligne de commande pour dialoguer avec l'API Kubernetes.
- **C'est quoi un DaemonSet ?** Un pod qui tourne sur **chaque** nœud (ex. l'agent de logs Promtail, node-exporter).

- **C'est quoi un CronJob ?** Une tâche planifiée récurrente (ex. la sauvegarde quotidienne de la base).

Réseau

- **Comment les pods communiquent-ils ?** Chaque pod a une IP ; le **CNI (Calico)** assure le routage entre eux, même sur des nœuds différents.
- **C'est quoi CoreDNS ?** Le DNS interne du cluster (résout les noms de services).
- **C'est quoi un reverse proxy / load balancer ?** Il répartit le trafic sur plusieurs pods et route selon l'hôte/le chemin (rôle de nginx-ingress).
- **C'est quoi un NodePort ?** Un port ouvert sur chaque nœud qui expose un service vers l'extérieur.

Stockage

- **C'est quoi un PV / PVC ?** Le PersistentVolume est un espace de stockage ; le PVC est la demande d'un pod. Les données survivent au redémarrage du pod.
- **C'est quoi une StorageClass ?** Le « type » de stockage qui permet de provisionner des volumes automatiquement.

Déploiement / CI-CD

- **C'est quoi Helm ?** Un « gestionnaire de paquets » pour Kubernetes ; un **chart** = un paquet paramétrable.
- **C'est quoi Kustomize ?** Un outil pour personnaliser des manifestes YAML par environnement (bases + overlays).
- **C'est quoi le GitOps ?** Décrire l'état voulu dans Git ; un outil (**ArgoCD**) applique et maintient cet état automatiquement.
- **C'est quoi un rolling update ?** Une mise à jour progressive : les pods sont remplacés petit à petit, sans coupure.
- **C'est quoi un rollback ?** Revenir à la version précédente qui fonctionnait.

Observabilité

- **C'est quoi une métrique ?** Une mesure chiffrée dans le temps (CPU, RAM, nombre de requêtes).
- **C'est quoi Prometheus ?** Il collecte et stocke les métriques du cluster.
- **C'est quoi Grafana ?** Il affiche métriques et logs sous forme de tableaux de bord.
- **C'est quoi un log ?** Un message texte émis par une application (trace d'un événement).
- **C'est quoi Loki ?** Il centralise les logs de tous les pods.
- **C'est quoi Alertmanager ?** Il envoie des alertes quand une condition de Prometheus est remplie.

Sécurité (bases)

- **C'est quoi HTTPS / TLS ?** Le chiffrement des échanges web ; un certificat prouve l'identité du site.
 - **C'est quoi un certificat / cert-manager ?** Un certificat authentifie un domaine ; cert-manager les émet/renouvelle automatiquement.
 - **C'est quoi un firewall / security group ?** Un filtre réseau qui autorise ou bloque des ports (sur AWS = security group).
 - **C'est quoi SSH ?** Un accès distant chiffré à une machine, via une clé.
 - **C'est quoi un Secret Kubernetes ?** Un objet qui stocke des données sensibles à part du reste de la config.
 - **C'est quoi RBAC ?** Le contrôle des permissions : qui a le droit de faire quoi dans le cluster.
 - **C'est quoi une NetworkPolicy ?** Une règle de pare-feu entre pods (qui peut parler à qui).
 - **C'est quoi un webhook d'admission (OPA) ?** Un contrôle qui valide ou refuse une requête à l'API avant qu'elle soit appliquée.
 - **C'est quoi le SSO / OIDC (dex, oauth2-proxy) ?** Une authentification centralisée : on se connecte une fois pour accéder aux outils.
-

A. Architecture & nœuds

Q1. Pourquoi 4 nœuds avec des rôles séparés (control-plane / worker / ingress / monitoring) ? Pour trois raisons : (1) **isolation des charges** — l'app, le trafic entrant et l'observabilité ne se disputent pas le CPU/RAM ; (2) **sécurité** — le nœud exposé à Internet (ingress) est séparé du cerveau du cluster (control-plane) ; (3) **lisibilité opérationnelle** — on sait où regarder en cas d'incident. Le placement est piloté par des labels de rôle et des `nodeSelector`.

Q2. Pourquoi kubeadm et pas un service managé (EKS) ? Le sujet demande de **construire** le cluster nous-mêmes pour maîtriser chaque composant (etcd, API server, scheduler, CNI). Un service managé cacherait le control-plane, ce qui est justement ce qu'on doit démontrer.

Q3. Que se passe-t-il si le control-plane (node1) tombe ? Les applications **déjà déployées continuent de tourner** (le kubelet de chaque nœud maintient ses pods). Ce qu'on perd temporairement : l'API (donc `kubectl`, l'auto-scaling, la réconciliation). C'est le point faible d'un control-plane à un seul nœud — en production on en mettrait 3 (etcd HA). Ici c'est un choix assumé lié aux 4 VM du sujet.

Q4. Pourquoi Calico comme CNI ? Réseau pod-à-pod robuste et surtout **support natif des NetworkPolicies**, dont on a besoin pour l'isolation réseau (exigence sécurité).

Q5. Pourquoi seul node1 a une IP Elastic (fixe) ? node1 est le point d'administration stable : tout `kubectl` passe par lui, il ne doit pas changer d'adresse aux redémarrages. Les autres nœuds n'ont pas besoin d'IP publique stable car on les gère via le control-plane.

B. Déploiement & GitOps

Q6. L'app est déployée avec Helm ou Kustomize ? Les deux, à deux niveaux : un **Helm chart** package l'application (valeurs paramétrables, chart officiel pour la base). **Kustomize** organise le dépôt GitOps (bases + overlays par environnement). **ArgoCD** synchronise le tout depuis Git.

Q7. Que fait ArgoCD exactement ? Il applique le **GitOps** : Git est la source de vérité. ArgoCD compare en continu l'état du cluster à ce qui est déclaré dans Git et **réconcilie** (selfHeal). Si quelqu'un modifie manuellement une ressource, ArgoCD la remet conforme à Git.

Q8. Certaines apps ArgoCD sont « OutOfSync » — pourquoi ? Ce sont des composants d'infrastructure que nous avons installés/ajustés manuellement pendant la mise au point (ex. cert-manager, monitoring). Ils ne sont pas encore alignés à 100 % avec les manifestes Git. L'app métier `laravel-app`, elle, est **Synced/Healthy**. ⚠ On ne force **pas** un sync global (notamment pas `infra-security`, voir Q25).

Q9. Montrez un déploiement complet, puis un déploiement cassé avec rollback automatique. Déploiement : `helm` / `kubectl apply` ou sync ArgoCD → pods Running. Rollback : on déploie une version cassée (probe qui échoue) → le **rolling update s'arrête** et garde les anciens pods (l'app reste UP), puis `kubectl rollout undo` revient à la version saine. On l'a vécu en vrai : une image sur le mauvais port a fait échouer les probes, les anciens pods ont continué à servir, aucune coupure.

C. Haute disponibilité & scaling

Q10. Comment fonctionne l'auto-scaling (HPA) ? Le HPA surveille le CPU (via metrics-server) et ajuste le nombre de réplicas entre **3 et 10** quand la charge dépasse **70 %**. En démo, on envoie beaucoup de requêtes et on voit les réplicas monter dans Grafana.

Q11. À quoi servent les règles d'anti-affinité ? Elles répartissent les réplicas sur des nœuds différents. Si un nœud tombe, il reste des réplicas ailleurs → pas de coupure.

Q12. Que se passe-t-il si un pod plante ? Le contrôleur le **recrée automatiquement** (ReplicaSet). Les probes de liveness détectent un pod bloqué et le redémarrent ; readiness le retire du service tant qu'il n'est pas prêt.

D. Persistance & sauvegardes

Q13. Comment les données de la base sont-elles persistées ? MySQL tourne en **StatefulSet** avec un **PersistentVolumeClaim** monté (le PV survit au redémarrage/reprogrammation du pod). Les données ne sont pas dans le conteneur éphémère.

Q14. Comment marchent les sauvegardes et où sont-elles stockées ? Un **CronJob** quotidien fait un dump MySQL dans un PVC dédié. On peut aussi la déclencher à la demande : `kubectl create job --from=cronjob/mysql-backup ...`. (Note honnête : la StorageClass manquait au départ et bloquait le PVC de backup ; on l'a corrigée pendant l'audit.)

E. Sécurité

Q15. Comment gérez-vous les secrets ? Via des **Secrets Kubernetes** injectés par `secretKeyRef` (pas de secret en clair dans un ConfigMap). Côté docker-compose, les identifiants sont sortis du fichier et lus depuis un `.env` non versionné. Notre audit a justement trouvé des secrets en clair au départ, qu'on a corrigés (PR #5/#6).

Q16. Que fait le validating webhook (OPA) ? Il **contrôle chaque requête de création** à l'API. On a une politique (via **Gatekeeper**, l'OPA officiel) qui refuse un pod sans le label obligatoire `app.kubernetes.io/name`. Démo : `kubectl run` sans label → **refusé** ; avec label → accepté.

Q17. Comment les NetworkPolicies protègent-elles l'app ? Elles imposent que **seuls l'ingress et le monitoring** peuvent joindre l'app ; tout autre namespace est bloqué. Démo : depuis `default` → timeout ; depuis `ingress-nginx` → 200.

Q18. Comment gérez-vous l'authentification aux outils ? Chaque outil a son auth (ArgoCD, Grafana, Dashboard par token). En plus, on a déployé un **SSO OIDC dex + oauth2-proxy** devant Prometheus : un accès non authentifié est **redirigé (302) vers la page de login dex**.

Q19. Comment avez-vous trouvé les vulnérabilités et qu'avez-vous corrigé ? Audit en 3 temps : (1) 7 axes d'analyse ; (2) vérification contradictoire de chaque trouvaille ; (3) **vérification en ligne de chaque CVE sur NVD**. Résultat : 23 confirmées, dont **un seul CVE applicable, CVE-2024-52301**, neutralisé en désactivant `register_argc_argv` (image reconstruite et déployée). On a aussi montré que 3 CVE souvent cités **ne s'appliquaient pas** à notre version.

F. Monitoring & logs

Q20. Que monitoriez-vous et comment ? **Prometheus** collecte les métriques (CPU/RAM/réseau/pods) de tout le cluster ; **Grafana** les visualise (28 dashboards pré-configurés) ; **Alertmanager** envoie les alertes. Un `node-exporter` tourne sur chaque nœud.

Q21. Comment fonctionne la collecte des logs ? **Loki** agrège les logs ; l'agent **Promtail** (DaemonSet) tourne sur chaque nœud, collecte les logs des pods et les envoie à Loki. On les consulte dans Grafana (source de données Loki), avec filtrage par namespace/label et recherche plein texte.

G. Accès, ingress, HTTPS

Q22. Pourquoi tout est en HTTP et pas en HTTPS ? Le sujet ne demande **pas** HTTPS : le chiffrement (Let's Encrypt + cert-manager) est listé comme **bonus**. HTTP suffit pour les exigences obligatoires. Cela dit, on a déjà déployé **cert-manager** et un HTTPS auto-signé fonctionne ; il ne manque que le certificat public Let's Encrypt, bloqué par une contrainte réseau du control-plane.

Q23. Comment le trafic externe est-il routé vers l'app ? Le trafic entre par le **contrôleur nginx-ingress** qui route **par nom d'hôte** vers le bon service. Chaque domaine (`larapp...`, `grafana...`, `argocd...`) pointe vers le même ingress qui aiguille en interne.

H. Questions pièges / autocritique

Q24. Quel est le point faible de votre architecture, et qu'amélioreriez-vous ? (1) **Control-plane à un seul nœud** (pas d'etcd HA) → en prod, 3 control-planes. (2) **Framework Laravel 8.x en fin de vie** → migration vers une version supportée. (3) **Image mono-étage avec deps de dev** → build multi-étages `--no-dev`. On assume ces choix, contraintes par les 4 VM et le temps ; on sait exactement quoi améliorer.

Q25. Pourquoi `imagePullPolicy: Never` ? Comment déployez-vous une nouvelle image ? Faute de registre privé, l'image est **construite puis chargée dans le containerd de chaque nœud** (`Never` = ne pas tenter de puller). Pour déployer une nouvelle version, on construit sur node1 (`nerdctl`), on **distribue l'image aux workers**, puis on met à jour le tag via ArgoCD (rolling update). C'est ce qu'on a fait pour l'image de sécurité `secure2`. Un vrai registre simplifierait ce point.

Q26. Le sujet demande de repartir d'un cluster neuf le jour de la défense — comment ? Scripts + manifestes : `kubeadm init` (control-plane) → join des workers → Calico → nginx-ingress → kube-prometheus → l'app via `helm` / `kubectl apply -f /kustomize` / ArgoCD. Tout est documenté et rejouable (voir le runbook de déploiement).